Database Management Systems

Practical Lab File

Submitted in partial fulfilment of the requirement of the degree of

BACHELOR OF TECHNOLOGY (FSD)

Semester-IV

to

Dr Preeti Rathi

Assistant Professor (SOET)

by

Adit Ghosh 2401350016

GitHub Link: https://github.com/adit-ghosh/dbms-practical-file-2401350016

INDEX

S No	Content	Page No
1.	Index	1
2.	To study and implement various DDL commands to create and modify a table in MySQL.	2
3.	To study and implement various DML commands to insert, retrieve, update, and delete data.	4
4.	To study and implement DCL commands to control access and permissions in MySQL.	6
5.	To implement aggregate functions MIN(), MAX(), SUM(), AVG(), and COUNT() on a table in MySQL.	8
6.	To perform data retrieval using conditions (greater than) and sorting techniques using ASC and DESC in MySQL.	10
7.	To implement NATURAL JOIN and EQUI JOIN on tables in MySQL.	12
8.	To implement LEFT OUTER JOIN and RIGHT OUTER JOIN on tables in MySQL.	14
9.	To implement SELF JOIN on a table in MySQL.	16

Experiment 1

Aim: To study and implement various DDL commands to create and modify a table in MySQL.

Query:

```sql
-- PRACTICAL 1: DDL COMMANDS
-- AIM: To study and implement various DDL commands to create and modify a table in MySQL.
DROP DATABASE IF EXISTS DBMS_Practicals;
CREATE DATABASE IF NOT EXISTS DBMS_Practicals;
USE DBMS_Practicals;

DROP TABLE IF EXISTS Employees;
CREATE TABLE IF NOT EXISTS Employees (
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(50) NOT NULL,
    Department VARCHAR(30),
    Salary DECIMAL(10, 2),
    ManagerID INT
);

DESCRIBE Employees;

ALTER TABLE Employees ADD DateOfJoin DATE;
ALTER TABLE Employees MODIFY EmpName VARCHAR(100);
ALTER TABLE Employees CHANGE COLUMN DateOfJoin DOJ DATE;
ALTER TABLE Employees DROP COLUMN DOJ;

-- Adding sample data to verify table functionality
INSERT INTO Employees (EmpID, EmpName, Department, Salary, ManagerID) VALUES
(101, 'Rahul Sharma', 'IT', 75000.00, NULL),
(102, 'Priya Verma', 'HR', 60000.00, 101),
(103, 'Amit Gupta', 'IT', 70000.00, 101);

SELECT * FROM Employees;
TRUNCATE TABLE Employees;
SELECT * FROM Employees;
DESCRIBE Employees;
```

Output:

Result Grid | Filter Rows: | Export: | Wrap C

Field	Type	Null	Key	Default	Extra
EmpID	int	NO	PRI	NULL	
EmpName	varchar(50)	NO		NULL	
Department	varchar(30)	YES		NULL	
Salary	decimal(10,2)	YES		NULL	
ManagerID	int	YES		NULL	

EmpID	EmpName	Department	Salary	ManagerID
101	Rahul Sharma	IT	75000.00	NULL
102	Priya Verma	HR	60000.00	101
103	Amit Gupta	IT	70000.00	101
NULL	NULL	NULL	NULL	NULL

EmpID	EmpName	Department	Salary	ManagerID
NULL	NULL	NULL	NULL	NULL

Result Grid | Filter Rows: | Export: | Wrap C

Field	Type	Null	Key	Default	Extra
EmpID	int	NO	PRI	NULL	
EmpName	varchar(100)	YES		NULL	
Department	varchar(30)	YES		NULL	
Salary	decimal(10,2)	YES		NULL	
ManagerID	int	YES		NULL	

Experiment 2

Aim: To study and implement various DML commands to insert, retrieve, update, and delete data.

Query:

```sql
-- PRACTICAL 2: DML COMMANDS
-- AIM: To study and implement various DML commands to insert, retrieve, update, and delete data.

-- 1. INSERT: Adding data into the table
INSERT INTO Employees (EmpID, EmpName, Department, Salary, ManagerID) VALUES
(101, 'Rahul Sharma', 'IT', 75000.00, NULL),
(102, 'Priya Verma', 'HR', 60000.00, 101),
(103, 'Amit Gupta', 'IT', 70000.00, 101),
(104, 'Suresh Lal', 'Finance', 55000.00, 102),
(105, 'Neha Kapoor', 'HR', 62000.00, 102);

-- 2. SELECT: Retrieving data
-- Retrieving all columns
SELECT * FROM Employees;

-- Retrieving specific columns with a condition
SELECT EmpName, Salary FROM Employees WHERE Department = 'IT';
```

```sql

-- 3. UPDATE: Modifying existing data
UPDATE Employees SET Salary = 80000.00 WHERE EmpID = 101;
UPDATE Employees SET Department = 'Operations' WHERE EmpID = 104;

-- Verifying updates
SELECT * FROM Employees;

-- 4. DELETE: Removing specific records
DELETE FROM Employees WHERE EmpID = 105;

-- Verifying deletion
SELECT * FROM Employees;
```

Output:

Result Grid | Filter Rows: | Edit:

EmpID	EmpName	Department	Salary	ManagerID
101	Rahul Sharma	IT	75000.00	NULL
102	Priya Verma	HR	60000.00	101
103	Amit Gupta	IT	70000.00	101
104	Suresh Lal	Finance	55000.00	102
105	Neha Kapoor	HR	62000.00	102
NULL	NULL	NULL	NULL	NULL

Result Grid | Filter Rows:

EmpName	Salary
Rahul Sharma	75000.00
Amit Gupta	70000.00

Result Grid | Filter Rows: | Edit:

EmpID	EmpName	Department	Salary	ManagerID
101	Rahul Sharma	IT	80000.00	NULL
102	Priya Verma	HR	60000.00	101
103	Amit Gupta	IT	70000.00	101
104	Suresh Lal	Operations	55000.00	102
105	Neha Kapoor	HR	62000.00	102
NULL	NULL	NULL	NULL	NULL

Result Grid | Filter Rows: | Edit:

EmpID	EmpName	Department	Salary	ManagerID
101	Rahul Sharma	IT	80000.00	NULL
102	Priya Verma	HR	60000.00	101
103	Amit Gupta	IT	70000.00	101
104	Suresh Lal	Operations	55000.00	102
NULL	NULL	NULL	NULL	NULL

Experiment 3

Aim: To study and implement DCL commands to control access and permissions in MySQL.

Query:

```sql
-- PRACTICAL 3: DCL COMMANDS
-- AIM: To study and implement DCL commands to control access and permissions in MySQL.

-- 1. Creating a new user for demonstration
CREATE USER IF NOT EXISTS 'test_user'@'localhost' IDENTIFIED BY 'password123';

-- 2. GRANT: Giving permissions to the user
-- Granting SELECT permission on the Employees table
GRANT SELECT ON DBMS_Practicals.Employees TO 'test_user'@'localhost';

-- 3. REVOKE: Taking back permissions
-- Revoking the SELECT permission
REVOKE SELECT ON DBMS_Practicals.Employees FROM 'test_user'@'localhost';

-- 4. Cleaning up
DROP USER 'test_user'@'localhost';
```

Output:

100	10:02:11	CREATE USER IF NOT EXISTS 'test_user'@'localhost' IDENTIFIED BY 'password123'	0 row(s) affected	0.015 sec
101	10:02:11	GRANT SELECT ON DBMS_Practicals.Employees TO 'test_user'@'localhost'	0 row(s) affected	0.000 sec
102	10:02:11	REVOKE SELECT ON DBMS_Practicals.Employees FROM 'test_user'@'localhost'	0 row(s) affected	0.016 sec
103	10:02:11	DROP USER 'test_user'@'localhost'	0 row(s) affected	0.016 sec

Experiment 4

Aim: To implement aggregate functions MIN(), MAX(), SUM(), AVG(), and COUNT() on a table in MySQL.

Query:

```sql
-- PRACTICAL 4: AGGREGATE FUNCTIONS
-- AIM: To implement aggregate functions MIN(), MAX(), SUM(), AVG(), and COUNT() on a table in MySQL.

-- 1. COUNT: Total number of employees
SELECT COUNT(*) AS TotalEmployees FROM Employees;

-- 2. SUM: Total salary paid by the company
SELECT SUM(Salary) AS TotalSalary FROM Employees;

-- 3. AVG: Average salary of employees
SELECT AVG(Salary) AS AverageSalary FROM Employees;

-- 4. MIN: Minimum salary in the table
SELECT MIN(Salary) AS MinimumSalary FROM Employees;

-- 5. MAX: Maximum salary in the table
SELECT MAX(Salary) AS MaximumSalary FROM Employees;
```

Output:

	TotalEmployees
▶	4

	TotalSalary
▶	265000.00

	AverageSalary
▶	66250.000000

	MinimumSalary
▶	55000.00

	MaximumSalary
▶	80000.00

Experiment 5

Aim: To perform data retrieval using conditions (greater than) and sorting techniques using ASC and DESC in MySQL.

Query:

```sql
-- PRACTICAL 5: GROUP BY AND HAVING CLAUSE
-- AIM: To implement GROUP BY and HAVING clauses to group rows and filter groups in MySQL.

-- 1. GROUP BY: Grouping employees by department
SELECT Department, COUNT(*) AS NumberOfEmployees
FROM Employees
GROUP BY Department;

-- 2. HAVING: Filtering groups with more than 1 employee
SELECT Department, COUNT(*) AS NumberOfEmployees
FROM Employees
GROUP BY Department
HAVING COUNT(*) > 1;

-- 3. Multiple aggregate functions
SELECT Department, COUNT(*) AS NumberOfEmployees, AVG(Salary) AS AverageSalary
FROM Employees
GROUP BY Department;
```

Output:

Department	NumberOfEmployees
IT	2
HR	1
Operations	1

Department	NumberOfEmployees
IT	2

Department	NumberOfEmployees	AverageSalary
IT	2	75000.000000
HR	1	60000.000000
Operations	1	55000.000000

Experiment 6

Aim: To implement NATURAL JOIN and EQUI JOIN on tables in MySQL.

Query:

```sql
-- PRACTICAL 6: NATURAL JOIN AND EQUI JOIN
-- AIM: To implement NATURAL JOIN and EQUI JOIN on tables in MySQL.

-- To perform joins, we need a second table.
CREATE TABLE IF NOT EXISTS Departments (
    Department VARCHAR(30) PRIMARY KEY,
    Location VARCHAR(50)
);

-- Inserting data into Departments
INSERT INTO Departments (Department, Location) VALUES
('IT', 'Floor 1'),
('HR', 'Floor 2'),
('Finance', 'Floor 3'),
('Operations', 'Floor 4');

-- 1. EQUI JOIN: Joining tables using a common column with '=' operator
SELECT Employees.EmpName, Employees.Department, Departments.Location
FROM Employees
JOIN Departments ON Employees.Department = Departments.Department;

-- 2. NATURAL JOIN: Joining tables automatically based on columns with same name
SELECT * FROM Employees NATURAL JOIN Departments;
```

Output:

	EmpName	Department	Location
▶	Rahul Sharma	IT	Floor 1
	Priya Verma	HR	Floor 2
	Amit Gupta	IT	Floor 1
	Suresh Lal	Operations	Floor 4

	Department	EmpID	EmpName	Salary	ManagerID	Location
▶	IT	101	Rahul Sharma	80000.00	NULL	Floor 1
	HR	102	Priya Verma	60000.00	101	Floor 2
	IT	103	Amit Gupta	70000.00	101	Floor 1
	Operations	104	Suresh Lal	55000.00	102	Floor 4

Experiment 7

Aim: To implement LEFT OUTER JOIN and RIGHT OUTER JOIN on tables in MySQL.

Code:

```sql
-- PRACTICAL 7: LEFT OUTER JOIN AND RIGHT OUTER JOIN
-- AIM: To implement LEFT OUTER JOIN and RIGHT OUTER JOIN on tables in MySQL.

-- Adding sample data to show unmatched records
INSERT INTO Departments (Department, Location) VALUES ('Marketing', 'Floor 5');
INSERT INTO Employees (EmpID, EmpName, Department, Salary, ManagerID) VALUES (106, 'Vicky Malhotra', NULL, 48000.00, 101);

-- 1. LEFT OUTER JOIN: Returns all records from the left table, and matched records from the right table.
-- Employees is Left, Departments is Right. This will show 'Vicky' even without a department.
SELECT Employees.EmpName, Employees.Department, Departments.Location
FROM Employees
LEFT JOIN Departments ON Employees.Department = Departments.Department;

-- 2. RIGHT OUTER JOIN: Returns all records from the right table, and matched records from the left table.
-- This will show 'Marketing' and 'Finance' even if they have no employees.
SELECT Employees.EmpName, Departments.Department, Departments.Location
FROM Employees
RIGHT JOIN Departments ON Employees.Department = Departments.Department;
```

Output:

EmpName	Department	Location
Rahul Sharma	IT	Floor 1
Priya Verma	HR	Floor 2
Amit Gupta	IT	Floor 1
Suresh Lal	Operations	Floor 4
Vicky Malhotra	NULL	NULL

EmpName	Department	Location
NULL	Finance	Floor 3
Priya Verma	HR	Floor 2
Amit Gupta	IT	Floor 1
Rahul Sharma	IT	Floor 1
NULL	Marketing	Floor 5
Suresh Lal	Operations	Floor 4

Experiment 8

Aim: To implement SELF JOIN on a table in MySQL.

Code:

```sql
-- PRACTICAL 8: SELF JOIN
-- AIM: To implement SELF JOIN on a table in MySQL.

-- SELF JOIN: Joining the 'Employees' table with itself to find managers
-- We use aliases 'e' for employee and 'm' for manager.
SELECT e.EmpName AS Employee, m.EmpName AS Manager
FROM Employees e
JOIN Employees m ON e.ManagerID = m.EmpID;

-- This query matches the ManagerID of an employee with the EmpID of another employee in the same table.
```

Output:

Employee	Manager
Priya Verma	Rahul Sharma
Amit Gupta	Rahul Sharma
Suresh Lal	Priya Verma
Vicky Malhotra	Rahul Sharma